

phylo: Technical Document

M. J. Wilson

September 2, 2026

Abstract

This document states the science behind **phylo**: the substitution model, the likelihood and the algorithm that evaluates it, the tree-search problem and its move sets, the reinforcement-learning formulation layered on top, and the computational structure that makes any of it tractable. It also holds the captions for the figures and tables the quality-assurance suite emits, so that a plot and its description cannot drift apart. Equations and algorithms follow the formulations in the sources cited throughout; deviations are stated where they occur.

Contents

1	Scope and status	2
2	Notation	2
3	The substitution model	2
3.1	Reversibility and the root distribution	3
3.2	The k -state Jukes–Cantor model	3
4	The likelihood	3
4.1	Pruning	4
4.2	Differentiable backends	4
4.3	Rate variation across sites	4
5	Simulation	5
6	Tree search	5
6.1	The size of the problem	5
6.2	Move sets	5
6.3	Canonical serialization	6
6.4	Extensions to the move set	6
6.5	Parsimony and likelihood	7
7	Gradients	7
8	Reinforcement-learning formulation	8
9	Computational structure	8
10	Quality assurance	8
11	Sources	13
11.1	Infrastructure: build, structure, and speed	13
11.2	Optimization: discrete and continuous	13
11.3	Application: the science	13

A Background material	15
A.1 NNI and SPR	15

Intended reader. Intended reader is: well-educated developer with scientific and performance-computing background, but not an expert in a given application, e.g. phylogenetics. The body stays streamlined — citing sources rather than re-deriving standard material inline — and pushes required application background into Appendix A, referenced from the point of use. It is a higher-level overview of the current best-known approach (simulation, models) in line with the roadmap, not an exhaustive record; detail, plots, and results for support are linked to dedicated supporting documents, e.g. the results of a dedicated study to find the best of a number of approaches.

1 Scope and status

The project’s goal is to search the space of phylogenetic tree topologies with reinforcement learning, scoring candidates by their likelihood under a model of character substitution. This document develops the theory that search requires. Most of it is not implemented yet. The k -state Jukes–Cantor substitution model, its simulator, and vectorized NumPy Felsenstein pruning (the Pruning subsection, `phylo.likelihood.pruning`) are; the rest of the repository is scaffolding — build, test, benchmark, lint, documentation, and audit pipelines around placeholder functions — and this document is written ahead of that implementation so it has a specification to match. Sections marked *not yet implemented* describe intent; everything else describes mathematics that does not depend on our code.

2 Notation

Let $\mathcal{A}$ be the finite state space of a single character, with $k = |\mathcal{A}|$; for nucleotide data $\mathcal{A} = \{A, C, G, T\}$ and $k = 4$. The data $\mathbf{D}$ are an alignment of n taxa over m sites, so $\mathbf{D} \in \mathcal{A}^{n \times m}$, with $\mathbf{D}_{\cdot s}$ the observed column at site s . A phylogeny is a pair $(\tau, \mathbf{t})$: a tree topology τ whose leaves carry the taxa, and a vector $\mathbf{t}$ of non-negative branch lengths, measured in expected substitutions per site. Internal nodes represent unobserved ancestors. Table 1 collects the symbols used throughout.

Symbol	Meaning
$\mathcal{A}, k$	character state space and its size
n, m	number of taxa, number of sites
τ	tree topology
$\mathbf{t}$	vector of branch lengths
$\mathbf{Q}$	instantaneous rate matrix, $k \times k$
$\mathbf{P}(t)$	transition-probability matrix over a branch of length t
π	root (and, under reversibility, stationary) distribution
$L_u(i)$	partial likelihood at node u given state i

Table 1: Symbols used in this document.

3 The substitution model

Character evolution along a branch is a continuous-time Markov chain on $\mathcal{A}$ with rate matrix $\mathbf{Q}$, whose off-diagonal entries $q_{ij} \geq 0$ give the instantaneous rate of substitution from state i to state

j and whose rows sum to zero,

$$q_{ii} = - \sum_{j \neq i} q_{ij}. \quad (1)$$

Transition probabilities over a branch of length t follow from the matrix exponential,

$$\mathbf{P}(t) = \exp(\mathbf{Q}t), \quad P_{ij}(t) = \Pr(X_t = j \mid X_0 = i). \quad (2)$$

$\mathbf{P}(t)$ is a stochastic matrix: $P_{ij}(t) \geq 0$ and each row sums to one. Both properties are invariants the implementation is tested against.

3.1 Reversibility and the root distribution

The models used here are time-reversible with respect to a distribution $\boldsymbol{\pi}$, meaning they satisfy detailed balance,

$$\pi_i q_{ij} = \pi_j q_{ji} \quad \text{for all } i, j. \quad (3)$$

Reversibility has two consequences we rely on. First, $\boldsymbol{\pi}$ is the stationary distribution of the chain, which makes it the natural choice for the distribution of states at the root. Second, the likelihood is invariant to where the root is placed on the tree — Felsenstein’s *pulley principle* [8] — so an unrooted topology suffices for inference, and re-rooting is a test the implementation must pass rather than a modelling decision. A general reversible rate matrix factorizes as

$$q_{ij} = s_{ij} \pi_j \quad (i \neq j), \quad s_{ij} = s_{ji}, \quad (4)$$

which is the general time-reversible (GTR) model; the familiar JC69, K80, and HKY85 models are constrained special cases [8]. Branch lengths measure expected substitutions per site, which fixes the scale of $\mathbf{Q}$ through the normalization

$$- \sum_i \pi_i q_{ii} = 1. \quad (5)$$

Without (5), $\mathbf{Q}$ and $\mathbf{t}$ trade off against each other and neither is identifiable.

3.2 The k -state Jukes–Cantor model

The most constrained case is worth stating in closed form, because it gives the simulator an oracle that owes nothing to our likelihood code. Take all exchange rates equal and $\boldsymbol{\pi}$ uniform on k states. Under the normalization (5), the transition probabilities over a branch of length t are

$$P_{ii}(t) = \frac{1}{k} + \frac{k-1}{k} e^{-kt/(k-1)}, \quad P_{ij}(t) = \frac{1}{k} - \frac{1}{k} e^{-kt/(k-1)} \quad (i \neq j). \quad (6)$$

At $t = 0$ this gives the identity, and as $t \rightarrow \infty$ every entry tends to $1/k$, the stationary distribution. For $k = 4$ it is JC69 [8]. Simulated substitution frequencies are checked against (6) within Monte Carlo error, which tests the simulator independently of the pruning implementation — if both were validated against each other, a shared error would pass unnoticed.

4 The likelihood

Sites are modelled as independent given $(\tau, \mathbf{t}, \mathbf{Q}, \boldsymbol{\pi})$, so the log-likelihood is a sum over columns,

$$\log \mathcal{L}(\tau, \mathbf{t}, \mathbf{Q}, \boldsymbol{\pi} \mid \mathbf{D}) = \sum_{s=1}^m \log \Pr(\mathbf{D}_{\cdot s} \mid \tau, \mathbf{t}, \mathbf{Q}, \boldsymbol{\pi}). \quad (7)$$

The site term marginalizes over the unobserved states of every internal node. Written directly, that is a sum over k^{n-1} assignments — intractable beyond a handful of taxa. Felsenstein’s pruning algorithm computes it in time linear in n by exploiting the tree’s conditional independence structure [8]; it is the sum-product algorithm on a tree [14, 12].

4.1 Pruning

Define the partial likelihood $L_u(i)$ as the probability of the data at the leaves below node u , given that u is in state i . At a leaf with observed state a , $L_u(i) = \mathbb{1}[i = a]$. At an internal node u with children v joined by branches of length t_v ,

$$L_u(i) = \prod_{v \in \text{children}(u)} \left(\sum_{j \in \mathcal{A}} P_{ij}(t_v) L_v(j) \right), \quad (8)$$

and at the root r the site likelihood is

$$\Pr(\mathbf{D}_{\cdot s} \mid \cdot) = \sum_{i \in \mathcal{A}} \pi_i L_r(i). \quad (9)$$

Algorithm 1 costs $O(nk^2)$ per site against $O(k^{n-1})$ for direct marginalization. The two agree

Algorithm 1 Felsenstein pruning for one site

Require: topology τ , branch lengths $\mathbf{t}$, rate matrix $\mathbf{Q}$, root distribution $\boldsymbol{\pi}$, observed column $\mathbf{D}_{\cdot s}$

Ensure: site likelihood $\Pr(\mathbf{D}_{\cdot s} \mid \tau, \mathbf{t}, \mathbf{Q}, \boldsymbol{\pi})$

```

1: for all nodes  $u$  in post-order do
2:   if  $u$  is a leaf with observed state  $a$  then
3:      $L_u(i) \leftarrow \mathbb{1}[i = a]$  for all  $i \in \mathcal{A}$ 
4:   else
5:      $L_u(i) \leftarrow \prod_{v \in \text{children}(u)} \sum_j P_{ij}(t_v) L_v(j)$  for all  $i \in \mathcal{A}$ 
6:   end if
7: end for
8: return  $\sum_i \pi_i L_r(i)$ 
```

exactly, which makes brute-force marginalization on small trees the reference implementation the pruning code is tested against. Partial likelihoods underflow for realistic m and n , so the implementation will rescale them per node and accumulate the log of the scaling factors. Rescaling changes the arithmetic but not the result: the rescaled and direct computations must agree to within floating-point tolerance on small problems where both run.

4.2 Differentiable backends

Branch lengths $\mathbf{t}$ are what gradient-based fitting and tree search differentiate through, so an accelerated backend must keep them in the autodiff graph rather than baking them into topology state. The PyTorch backend (`pruning_torch.py`, float64, CPU) takes $\mathbf{t}$ as a tensor separate from the topology and is accepted when its log-likelihood agrees with the NumPy reference to $\text{atol} = 10^{-9}$ – the same bound as the brute-force check above – never relaxed to accommodate a discrepancy. Gradients obtained from `torch.autograd` are checked against central finite differences of the NumPy oracle (step $\epsilon = 10^{-6}$) to $\text{atol} = 10^{-6}$, the coarser bound set by the finite-difference truncation and rounding error at that step size, not by the autodiff computation itself.

4.3 Rate variation across sites

Sites evolve at different rates. The standard treatment draws a site-specific rate multiplier r from a discretized gamma distribution with mean one, and averages the site likelihood over the c rate categories,

$$\Pr(\mathbf{D}_{\cdot s} \mid \cdot) = \frac{1}{c} \sum_{g=1}^c \Pr(\mathbf{D}_{\cdot s} \mid \tau, r_g \mathbf{t}, \mathbf{Q}, \boldsymbol{\pi}), \quad (10)$$

multiplying the cost of a likelihood evaluation by c [8]. *Not yet implemented.*

5 Simulation

Simulation is how the implementation is tested: draw data from known parameters, then check that each component recovers what the generating model implies. Generation runs in pre-order, the reverse of pruning. Every draw uses an explicitly seeded generator, so a simu-

Algorithm 2 Simulating an alignment under $(\tau, \mathbf{t}, \mathbf{Q}, \boldsymbol{\pi})$

Require: topology τ , branch lengths $\mathbf{t}$, rate matrix $\mathbf{Q}$, root distribution $\boldsymbol{\pi}$, sites m , seeded generator

Ensure: alignment $\mathbf{D} \in \mathcal{A}^{n \times m}$

```

1: for  $s \leftarrow 1$  to  $m$  do
2:   draw the root state  $x_r \sim \boldsymbol{\pi}$ 
3:   for all nodes  $u$  in pre-order,  $u \neq r$  with parent  $p$  and branch  $t_u$  do
4:     draw  $x_u \sim \mathbf{P}(t_u)_{x_p, \cdot}$ 
5:   end for
6:   record  $x_u$  at every leaf  $u$  as column  $s$ 
7: end for
8: return  $\mathbf{D}$ 

```

lated dataset is reproducible from its seed. Implemented for the k -state Jukes–Cantor case in `phylo.sim` (`simulate.py`), with topology, branch lengths, k , $\boldsymbol{\pi}$, seed, and site count read from a `simulation_params.yaml` and the generating parameters retained alongside the alignment. Simulated substitution frequencies are checked against (6) within a yamtl-declared Monte Carlo tolerance in `tests/regression/test_jc_simulate.py`. Table 3 in Section 10 lists every simulation regression fixture at the problem sizes actually run, read directly from each fixture’s yamtl; Figure 2 there shows a complete generated instance at the smallest of them — the Newick string and the leading simulated sites of the alignment.

6 Tree search

6.1 The size of the problem

The number of distinct unrooted binary topologies on n taxa is the double factorial $(2n - 5)!!$ [24], which grows faster than exponentially (Table 2). Exhaustive search is impossible past a handful of taxa, and finding the optimal tree under either the parsimony or the likelihood criterion is NP-hard [8]. Practical inference is therefore heuristic: start somewhere, propose rearrangements, keep what scores better.

Taxa n	Unrooted binary topologies $(2n - 5)!!$
10	2.0×10^6
20	2.2×10^{20}
50	2.8×10^{74}

Table 2: Growth of tree space with the number of taxa.

6.2 Move sets

Two rearrangement neighbourhoods define the search graph [8]: NNI and SPR, defined in Appendix A. NNI’s neighbourhood is small, cheap, and local; SPR’s is quadratic in n , so each step costs more, but its larger reach escapes local optima that trap NNI. Since neighbouring topologies differ only near the affected edges, most partial likelihoods are unchanged between a

tree and its neighbour. Caching and recomputing only the invalidated path is what makes tree search affordable, and is a design requirement on the likelihood engine rather than an optimization to add later.

6.3 Canonical serialization

Newick notation is not a unique encoding: one topology admits many strings, differing by the order in which children are written and by where the string is rooted. A search that keys on the raw string therefore fails to recognize a tree it has already scored, and a model trained on raw strings learns to distinguish spellings that carry no phylogenetic meaning. Algorithm 3 removes both problems by fixing a representative. Rooting at the smallest label is well defined

Algorithm 3 Canonical form of an unrooted binary tree with labelled leaves

Require: tree τ with leaves labelled from a totally ordered set

Ensure: a string identifying τ 's topology uniquely

```

1: root  $\tau$  at the leaf whose label is smallest in that order
2: for all nodes  $u$  in post-order do
3:   if  $u$  is a leaf then
4:      $\kappa(u) \leftarrow \text{label}(u)$ 
5:   else
6:     sort the children  $v$  of  $u$  by  $\kappa(v)$ 
7:      $\kappa(u) \leftarrow$  the concatenation of  $\kappa(v)$  in that order
8:   end if
9: end for
10: return  $\kappa(\text{root})$ , emitting children in sorted order

```

for any labelled tree, and the child order is fixed bottom-up by keys that depend only on the subtree below, so the output depends on the topology alone. Hashing the keys keeps the cost at $O(n \log n)$ [6]. Branch lengths are carried separately, as a vector in canonical node order, so one topology under two length assignments shares a topology key — which is what makes the key usable for memoizing likelihood evaluations across a search. Two properties are required of any implementation, and both are tested rather than assumed: two trees produce the same string exactly when they are the same topology, checked by randomly re-rooting and permuting the children of one tree and by comparing against distinct trees; and parsing a canonical string reproduces the tree that produced it.

6.4 Extensions to the move set

Three extensions target the same bottleneck: the number of likelihood evaluations a search spends. All three are proposals. *Not yet implemented.*

Bounding whole sets of trees. Scoring every neighbour exactly wastes work when most are poor. A cheap *upper* bound on the likelihood attainable anywhere within a set of topologies lets the search discard the whole set, in the manner of branch and bound [6]: when the bound falls below the best log-likelihood already achieved, no member of the set can win. Two sources of bounds are worth trying: relaxations of the pruning recursion (8), which replace the product over children by a quantity that is cheaper and provably no smaller; and compressed summaries of the alignment, where identical site patterns already collapse and coarser encodings trade tightness for speed [2]. Soundness is the whole game — a bound that can fall below an attainable likelihood silently discards the optimum — so each candidate bound needs a proof, and a test that searches small trees exhaustively for a counterexample.

Multi-moves. Composing several NNI or SPR rearrangements into one action reaches topologies no single move reaches, at the cost of a much larger action space; in reinforcement-learning terms these are temporally extended actions [25]. Rather than fixing how many rearrangements a compound move contains, draw the composition during training from a Dirichlet process, so the number of distinct compound moves the agent maintains grows with the evidence for them instead of being set in advance [14]. The measure of success is unchanged: fewer likelihood evaluations for the same likelihood reached.

A transformer policy over Newick strings. A topology serializes to a Newick string, which a transformer can consume directly — tokenize the string, pretrain on simulated trees, then fine-tune the policy with the usual policy-gradient machinery [23, 13, 25]. The input is the canonical form of Section 6.3, which settles the encoding question: one topology, one string. The alternative, an architecture invariant to child order and root placement, buys the same property at greater cost and without supplying the memoization key the search needs anyway. Compression then applies on top, replacing recurring subtrees by dictionary symbols — an extended alphabet over the canonical string [2] — which shortens the token sequence a transformer must attend over.

6.5 Parsimony and likelihood

The *large parsimony* problem asks for the topology minimizing the number of state changes needed to explain the data. We keep the search problem and replace the criterion: candidates are scored by the maximized log-likelihood of (7), with the continuous parameters $(\mathbf{t}, \mathbf{Q}, \boldsymbol{\pi})$ fitted per topology. Likelihood is the more principled criterion — it models the substitution process rather than counting changes, and it is statistically consistent where parsimony can be positively misleading [8] — at the cost of a far more expensive score.

7 Gradients

Fitting $(\mathbf{t}, \mathbf{Q}, \boldsymbol{\pi})$ for a candidate topology is a continuous optimization, and gradients make it tractable. Differentiating (2) with respect to a branch length gives

$$\frac{\partial \mathbf{P}(t)}{\partial t} = \mathbf{Q} \exp(\mathbf{Q}t) = \mathbf{Q} \mathbf{P}(t). \quad (11)$$

For a reversible $\mathbf{Q}$ with eigendecomposition $\mathbf{Q} = \mathbf{U} \boldsymbol{\Lambda} \mathbf{U}^{-1}$, both the exponential and its derivative are diagonal in the eigenbasis,

$$\mathbf{P}(t) = \mathbf{U} e^{\boldsymbol{\Lambda} t} \mathbf{U}^{-1}, \quad \frac{\partial \mathbf{P}(t)}{\partial t} = \mathbf{U} \boldsymbol{\Lambda} e^{\boldsymbol{\Lambda} t} \mathbf{U}^{-1}, \quad (12)$$

so a single decomposition serves every branch. Gradients of the log-likelihood with respect to all branch lengths follow from reverse-mode automatic differentiation through the pruning recursion (8), at a cost proportional to one forward evaluation [10, 22]. The optimizer consuming them is a standard constrained quasi-Newton or trust-region method [19]; the parameters carry constraints — non-negative branch lengths, a normalized $\boldsymbol{\pi}$ — so the geometry of the statistical manifold, and the natural gradient it induces, is the principled setting [1]. Analytic and automatic gradients are checked against central finite differences,

$$\frac{\partial f}{\partial \theta} \approx \frac{f(\theta + h) - f(\theta - h)}{2h}, \quad (13)$$

with h chosen to balance truncation against round-off. This check is a required test for any differentiable component. *Not yet implemented.*

8 Reinforcement-learning formulation

Classical heuristics propose moves by a fixed, hand-designed rule. We ask whether a learned policy proposes better ones, and cast the search as a Markov decision process $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ [25]:

State. The current topology with its fitted continuous parameters, together with a summary of the alignment.

Action. A move drawn from the NNI or SPR neighbourhood of the current topology.

Transition. Applying the move and refitting the continuous parameters — deterministic given the action, up to the optimizer.

Reward. The improvement in maximized log-likelihood, $R = \log \mathcal{L}(\tau') - \log \mathcal{L}(\tau)$.

Episode. One search trajectory from a starting tree, terminating on a step budget or when no move improves the score.

The natural performance measure is not the likelihood reached but the likelihood reached *per likelihood evaluation*: every method converges given unbounded evaluations, so the comparison against a hill-climbing baseline has to be made at equal budget. *Not yet implemented.*

9 Computational structure

Likelihood evaluation dominates the run time: every proposed move costs at least one evaluation, and search proposes many. Its structure is favourable — sites are independent by (7), and the per-node work in (8) is a small dense matrix–vector product — so the computation is a batched, data-parallel reduction over a tree. The project’s performance rule follows from that. Where a kernel plausibly achieves an order of magnitude over the vectorized NumPy reference at the sizes actually run, it targets the GPU through PyTorch, Triton, or JAX, which supply autodiff and batching alongside the parallelism; otherwise the work goes to the Rust backend. The threshold is a measured benchmark, not an expectation, and every accelerated kernel keeps its reference implementation as a pinned oracle. Kernel-level considerations — memory hierarchy, occupancy, parallel reduction — follow Hwu et al. [11]; the CPU-side counterparts, caching and where time actually goes, follow Bryant and O’Hallaron [4], and the backend itself is Rust [3]. Memory is the constraint to watch: partial likelihoods occupy $O(nmkc)$ floating-point values for n taxa, m sites, k states, and c rate categories, before any caching of unchanged subtrees across proposals.

10 Quality assurance

The test suite is expected to establish scientific validity, not to exercise syntax: fixtures come from component-wise simulation under known parameters, expected values come from analytic or brute-force sources, and the invariants stated above — stochasticity of $\mathbf{P}(t)$, detailed balance (3), agreement of pruning with direct marginalization, gradients against finite differences (13), re-rooting invariance — are checked directly. Part of that suite emits scientific-style figures and tables: parameter recovery against simulated truth, likelihood surfaces, convergence traces, and timing comparisons. **Their captions belong in this section**, added in the same change that adds the figure, so that a plot and its description cannot drift apart. Figure 1 is the first: the topology and branch lengths `phylo.sim.simulate` draws an alignment over, rendered by `phylo.qa.sim_tree` from the same 8-taxon regression fixture as `tests/regression/test_jc.simulate.py`. `infra/build_technical.doc.sh` regenerates the figure and the caption above — read verbatim from the file the script writes alongside it — on every build, so the two cannot drift apart. Table 3 tabulates the taxa count, site count, seed, and Monte Carlo tolerance of every simulation regression fixture, rendered

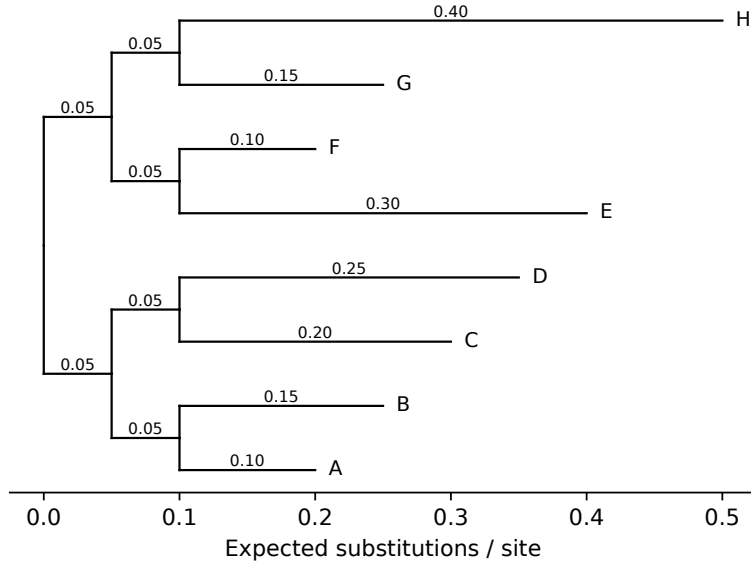


Figure 1: Simulated topology for 8 taxa under the Jukes-Cantor model (seed 20260904), branch lengths labelled in expected substitutions per site.

by `phylo.qa.sim_problem_sizes` directly from the `threesimulation_params.yaml` files rather than restated by hand, so the numbers in this document cannot drift from what the regression suite actually runs. Figure 2 shows a complete generated instance at the smallest of those

Fixture	Taxa	Sites	Seed	Tolerance
<code>simulation_params.yaml</code>	4	200000	20260902	0.01
<code>simulation_params_small_sites.yaml</code>	4	20000	20260903	0.03
<code>simulation_params_8taxa.yaml</code>	8	200000	20260904	0.01

Table 3: Problem-size parameters across the 3 simulation regression fixtures backing the Jukes-Cantor simulation test: taxa count, site count, seed, and the Monte Carlo tolerance each validates simulated substitution frequencies within.

sizes: the Newick string `phylo.sim.newick.to.newick` produces for the 4-taxon fixture, and the leading simulated sites of the alignment, nucleotide-coded for this $k = 4$ model, rendered by `phylo.qa.sim_example`. Figure 3 closes the simulation section by checking the simulator against the model it is supposed to realize. The curves are the closed-form transition probabilities of eq. (6); the markers are the substitution frequencies `phylo.sim.simulate` actually produced, one pair per branch. Agreement here is what licenses using simulated data as ground truth everywhere else in this document: the simulator is validated against the analytic form, never against the pruning code it feeds. Figure 4 turns to the likelihood engine, and to the two claims it rests on. Panel (a) checks every backend against brute-force marginalization over internal states,

(A:0.1,B:0.25,(C:0.15,D:0.4)ancestor_CD:0.05)root;

	0	1	2	3	4	5	6	7	8	9
A	T	G	A	T	G	G	G	T	A	G
B	A	G	T	T	T	T	G	T	A	G
C	T	G	A	T	T	G	G	T	A	G
D	G	G	A	T	C	T	G	C	T	A

Figure 2: Worked example: 4-taxon alignment simulated under the Jukes-Cantor model (seed 20260902, 200000 sites total), showing the generated Newick topology and the first 10 of 200000 simulated sites.

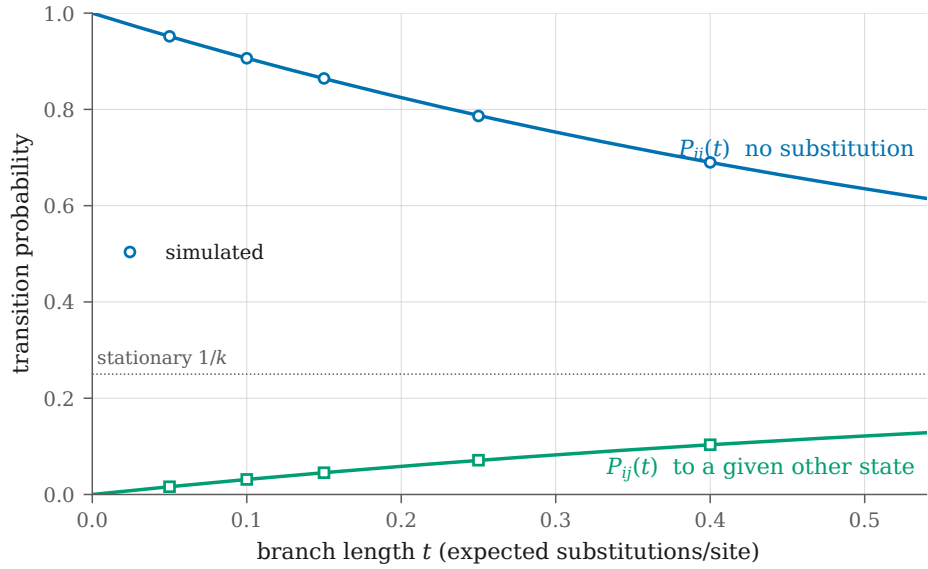


Figure 3: Closed-form Jukes-Cantor transition probabilities for $k = 4$ (curves) against the frequencies phylo.sim.simulate produced (markers), one pair per branch of the 5-branch fixture tree. Simulated at seed 20260902 over 200000 sites. Largest deviation $1.07\text{e-}03$, within the fixture's declared Monte Carlo tolerance of $1\text{e-}02$. The simulator is validated against this analytic form, never against the likelihood code it feeds.

an exhaustive sum that is a different algorithm rather than a second opinion from the same recursion; panel (b) checks the accelerated backends against the vectorized NumPy reference at the full fixture sizes, where that sum is no longer tractable. Panel (b) reports a *relative* deviation, and the reason generalizes. The total log-likelihood is a sum over sites, so its magnitude grows with the site count and so does any absolute discrepancy in it. The regression suite pins absolute tolerances at deliberately small sizes, where they are achievable; the same agreement, measured at 2×10^5 sites, reads several orders of magnitude larger in absolute terms while sitting at the `float64` floor relatively. An absolute tolerance fixed at one problem size does not transfer to another, which is a constraint on how the cross-device tolerance of Section 10 should eventually be stated. Figure 5 checks the derivative the continuous optimization of Milestone 4 will descend.

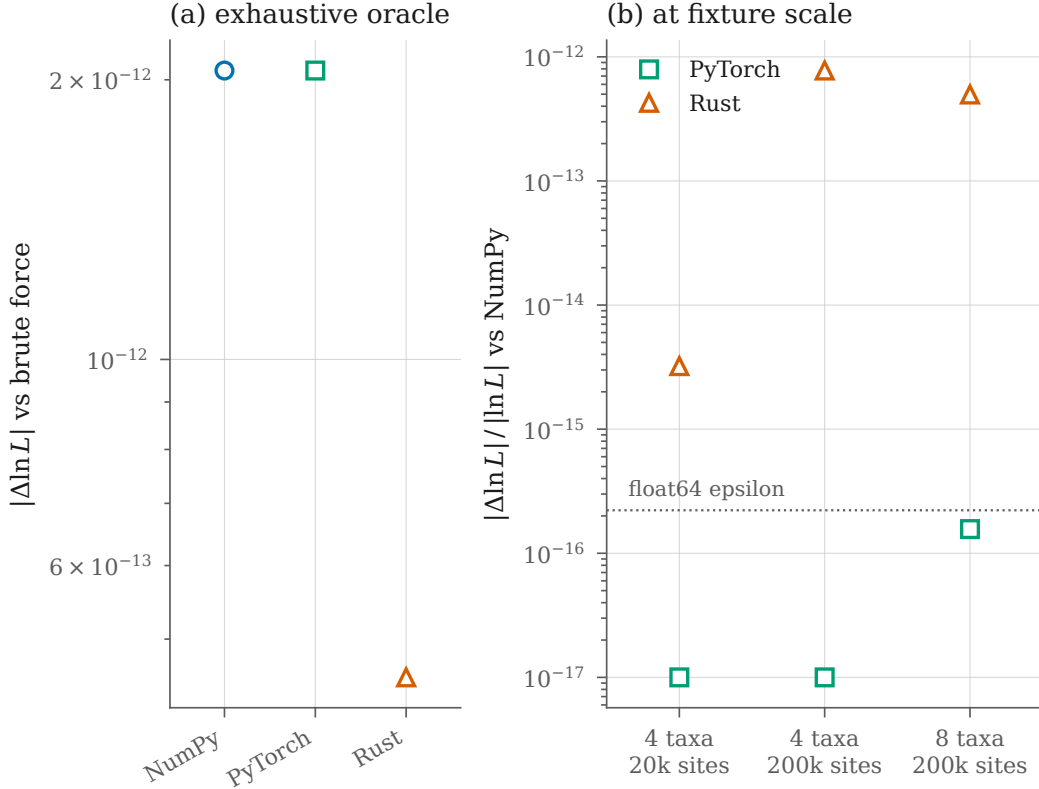


Figure 4: Backend agreement. (a) All three backends against brute-force marginalization over internal states on a 4-taxon tree at 400 sites, where the exhaustive sum is tractable and is an independent algorithm rather than a second opinion from the same recursion; largest deviation $2.05\text{e-}12$, within the $1\text{e-}09$ absolute tolerance the regression tests pin at these sizes. (b) The accelerated backends against the vectorized NumPy reference at full fixture sizes, where brute force is not available, plotted as relative deviation; largest $7.76\text{e-}13$, at the `float64` floor (dotted). Relative rather than absolute because the total log-likelihood is a sum over sites: the same agreement reads as $7.39\text{e-}07$ absolute at these sizes, so an absolute tolerance fixed at small sizes does not transfer. Points at the axis floor are exact agreement.

Autograd differentiates the PyTorch pruning recursion analytically; central finite differences of the independent NumPy likelihood approximate the same quantity numerically. Panel (b) sweeps the finite-difference step to show why the step must be stated rather than assumed: truncation error falls as the step shrinks while floating-point cancellation grows, so the disagreement is U-shaped in the step and the check means something only near its minimum. Search and optimization-trajectory QA figures are not built yet; this section is where their captions go once they are.

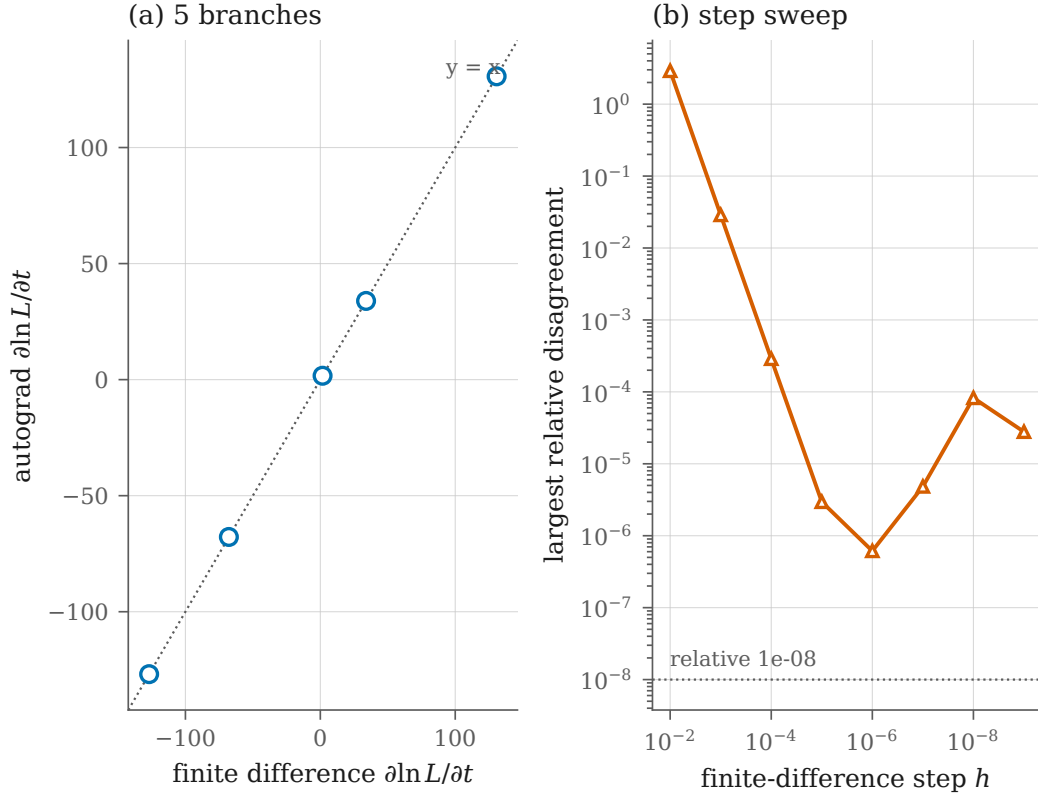


Figure 5: Analytic gradients against central finite differences, over the 5 branches of the fixture tree. (a) Autograd through the PyTorch pruning recursion against finite differences of the independent NumPy likelihood, at step $h = 1e-06$; points lie on $y = x$, largest relative disagreement $6.14e-07$. Two implementations and two methods, so agreement is evidence rather than self-consistency. (b) The same disagreement swept over h , x axis reversed so h decreases rightwards. The curve is U-shaped: truncation error falls as h shrinks while floating-point cancellation grows, best here at $h = 1e-06$ ($6.14e-07$). A finite-difference check is only meaningful near that minimum, which is why the step is stated. Disagreement is reported relative to the gradient magnitude: the regression tests pin an absolute $1e-06$ at 30 sites, but the gradient scales with the site count, so the same agreement reads as $1.03e-06$ absolute here and an absolute bound fixed at one size does not transfer.

11 Sources

The texts this project works from, grouped as `CLAUDE.md` groups them— a routing table keyed on what each informs, rather than a reading list. Where an algorithm we implement appears in one of them, this document follows its formulation and notation and says where it deviates.

11.1 Infrastructure: build, structure, and speed

Software craft. Martin [15] on the structure that keeps a change reviewable; Blandy et al. [3] on ownership, lifetimes, and the FFI boundary the Rust backend crosses.

Systems and hardware. Bryant and O’Hallaron [4] on the memory hierarchy and where CPU time goes; Hwu et al. [11] on GPU kernels, occupancy, and parallel reduction and scan.

11.2 Optimization: discrete and continuous

Algorithms and discrete mathematics. Cormen et al. [6] on data structures, traversal, complexity, and branch and bound; Rosen [24] on the counting and graph theory behind tree space.

Numerical optimization. Nocedal and Wright [19] on line search, trust region, and quasi-Newton methods for the continuous parameters.

Probabilistic inference. MacKay [14] on inference, message passing, and Monte Carlo; Koller and Friedman [12] on factor graphs and the exact-inference machinery that (8) instantiates; Frey [9] on belief propagation read across inference and coding; Ortega [20] on spectral methods for signals indexed by graph vertices.

Statistical physics. Mézard and Montanari [16] on inference over sparse graphs and phase transitions in hard combinatorial problems — tree search being one; Newman and Barkema [17] on sampling, equilibration, and error estimates from correlated samples, which the benchmark harness needs to rank variants rather than noise.

Information geometry. Amari [1] on the statistical manifold, the Fisher metric, and natural gradient.

Learning and reinforcement learning. Goodfellow et al. [10] and Prince [22] on architectures, optimization, and automatic differentiation; Sutton and Barto [25] on MDPs, value and policy methods, and temporally extended actions; Lapan [13] on implementation practice; Raschka [23] on transformer internals and tokenization, for the policy over canonical Newick strings proposed in Section 6.4.

11.3 Application: the science

Phylogenetics. Felsenstein [8] on substitution models, pruning, tree search, and the parsimony and likelihood criteria; Durbin et al. [7] on probabilistic models of sequences; Compeau and Pevzner [5] on the algorithmic view of alignment and phylogeny construction; Pachter and Sturmfels [21] on the algebraic structure of phylogenetic models and their invariants, which offer topology tests that avoid likelihood optimization altogether.

Information and quantum. Blahut [2] on algebraic coding, which is the background for the compressed tree encodings in Section 6.4; Nielsen and Chuang [18] on quantum information and algorithms, background only — nothing here targets quantum hardware.

References

- [1] Shun-ichi Amari. *Information Geometry and Its Applications*. Springer, Tokyo, 2016.
- [2] Richard E. Blahut. *Algebraic Codes for Data Transmission*. Cambridge University Press, Cambridge, 2003.
- [3] Jim Blandy, Jason Orendorff, and Leonora F. S. Tindall. *Programming Rust: Fast, Safe Systems Development*. O'Reilly Media, 2nd edition, 2021.
- [4] Randal E. Bryant and David R. O'Hallaron. *Computer Systems: A Programmer's Perspective*. Pearson, 3rd edition, 2015.
- [5] Phillip Compeau and Pavel Pevzner. *Bioinformatics Algorithms: An Active Learning Approach*. Active Learning Publishers, 2015.
- [6] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, Massachusetts, 4th edition, 2022.
- [7] Richard Durbin, Sean R. Eddy, Anders Krogh, and Graeme Mitchison. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press, Cambridge, 1998.
- [8] Joseph Felsenstein. *Inferring Phylogenies*. Sinauer Associates, Sunderland, Massachusetts, 2004.
- [9] Brendan J. Frey. *Graphical Models for Machine Learning and Digital Communication*. MIT Press, Cambridge, Massachusetts, 1998.
- [10] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, Massachusetts, 2016.
- [11] Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 4th edition, 2022.
- [12] Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, Cambridge, Massachusetts, 2009.
- [13] Maxim Lapan. *Deep Reinforcement Learning Hands-On*. Packt Publishing, 2nd edition, 2020.
- [14] David J. C. MacKay. *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, Cambridge, 2003.
- [15] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
- [16] Marc Mézard and Andrea Montanari. *Information, Physics, and Computation*. Oxford University Press, Oxford, 2009.
- [17] M. E. J. Newman and G. T. Barkema. *Monte Carlo Methods in Statistical Physics*. Oxford University Press, Oxford, 1999.
- [18] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, Cambridge, 10th anniversary edition, 2010.
- [19] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006.
- [20] Antonio Ortega. *Introduction to Graph Signal Processing*. Cambridge University Press, Cambridge, 2022.

- [21] Lior Pachter and Bernd Sturmfels, editors. *Algebraic Statistics for Computational Biology*. Cambridge University Press, Cambridge, 2005.
- [22] Simon J. D. Prince. *Understanding Deep Learning*. MIT Press, Cambridge, Massachusetts, 2023.
- [23] Sebastian Raschka. *Build a Large Language Model (From Scratch)*. Manning Publications, 2024.
- [24] Kenneth H. Rosen. *Discrete Mathematics and Its Applications*. McGraw-Hill, 8th edition, 2019.
- [25] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, Massachusetts, 2nd edition, 2018.

A Background material

Standard algorithms that are not specific to `phylo`, referenced from the main text rather than re-derived there (see the intended-reader note after the table of contents).

A.1 NNI and SPR

Two rearrangement neighbourhoods define the tree-search graph used in Section 6.4 and throughout [8]:

NNI (nearest-neighbour interchange). Each internal edge of an unrooted binary tree separates four subtrees, which can be rejoined in three ways; two are new topologies. With $n - 3$ internal edges, the NNI neighbourhood contains $2(n - 3)$ topologies — small, cheap, and local.

SPR (subtree prune and regraft). Detach a subtree and reattach it at another edge. The neighbourhood is quadratic in n , so each step costs more, but the larger reach escapes local optima that trap NNI.